



Payload Decoding Guide

MeteoWind IoT Pro Gen2 HNWL | How to parse the payloads
Firmware 1.02.004

1. What this document is for

This is a decoder-writer's guide for the MeteoWind IoT Pro Gen2 **HNWL** variant. It covers every LoRaWAN payload the station sends, in the order you need them to write a parser: how the bits are laid out, how each field converts to a physical value, and the five encodings that produce wrong readings if you decode them naively. A complete, tested JavaScript decoder is included in section 7.

The field layouts were taken from the firmware itself and the decoder in section 7 was verified against it and against the company's own bit-string calculator, so where this guide disagrees with an older payload sheet, this guide describes what firmware 1.02.004 actually does.

1.1 What this document covers

Port	Message	Size	Direction	Section
1	Periodic measurement	20 B	uplink	4
2	Service message MW1S - device identity	12 B	uplink	5.1
2	Service message MW2S - alarm settings readback	12 B	uplink	5.2
2	Service message MW0S - position, not implemented	12 B	uplink	5.3
50	Reply to a configuration command	varies	uplink	6.4
100-120	Configuration downlinks	varies	downlink	6.4

That is the whole protocol. Ports 3, 4 and 10 - the alarm uplink, the retransmission uplink and the alarm-settings downlink of the standard MeteoWind Gen2 - are compiled out of this firmware and never appear. Section 6 sets out what is missing and how a decoder should react if it sees one of them.

Port 50 carries short replies to configuration commands. They are bare hex fields with no message identifier, so they can only be interpreted against the command that was just sent, and they are outside the scope of this guide.

1.2 This is not the standard MeteoWind Gen2 payload

The HNWL periodic payload shares a port number with the standard MeteoWind IoT Pro Gen2 payload and nothing else. It is 20 bytes rather than 14, and every field after the index differs. Pointing a standard Gen2 decoder at an HNWL station produces plausible-looking wind data that is entirely wrong.

	Standard MeteoWind Gen2	MeteoWind Gen2 HNWL
Port 1 length	14 bytes	20 bytes
What it carries	One period average, plus 3 s and 1 s gusts, minimum, standard deviations, direction sector and turbulence	Eight 15-second sub-period averages of wind speed and wind direction, and nothing else
Uplink period	2 to 10 minutes, settable by downlink 132	Fixed at 2 minutes
Battery	1 bit, meaningful only across five consecutive indices	2 bits, a voltage bin in every payload

	Standard MeteoWind Gen2	MeteoWind Gen2 HNWL
Wind speed	12 bits at 0.02 Hz, one value	10 bits at 0.1 Hz, eight values
Wind direction	9 bits at 1 deg, average plus gust, sector and stdev	8 bits at 2 deg, eight values
Alarms	Port 3 uplink, port 10 downlink	Not implemented - see 6.1 and 6.3
Retransmission	Port 4 uplink, downlinks 108 and 109	Not implemented - see 6.2

The two products also differ in what a decoder can infer. The standard payload reports gusts and a direction sector, which HNWL does not; HNWL reports the shape of the wind within the period at 15-second resolution, which the standard payload does not. Neither can be derived from the other.



2. The parsing model

Every payload is one continuous bitstream. Fields are packed most-significant-bit first, back to back, with no padding or alignment between them, so most fields straddle byte boundaries. Only the index field happens to start and end on a byte.

Do not try to parse these payloads by slicing bytes. A byte-oriented parser works for the index field and then silently produces nonsense for everything after it. Read bit ranges.

2.1 Four steps

Step	What to do
1	Take the raw bytes and treat them as one bitstream, bit 1 being the most significant bit of byte 0
2	For each field in table order, read its width in bits, most significant bit first, into an unsigned integer - call it raw
3	If every bit of the field is set, the reading is INVALID. Emit null and stop processing that field. This applies to wind speed only - see 3.1
4	Otherwise the field value is raw x resolution + minimum, using the resolution and minimum from the field table

Step 4 gives the transmitted quantity, which for wind speed is a click frequency in Hz and not a speed. One further step turns it into a wind speed: apply the anemometer calibration of section 3.4.

2.2 A payload decoded by hand

Periodic payload 0385416C6B1EC7621C9824ABAC2CC08180ECEBC0 on port 1. Writing the 20 bytes out as 160 bits and cutting them at the field widths from section 4 gives:

Bits	Field	Bits read	raw	Value	Unit
1-8	Index	00000011	3	3	-
9-10	Battery bin	10	2	2	-
11-20	Wind speed 1	0001010100	84	8.4	Hz
21-30	Wind speed 2	0001011011	91	9.1	Hz
31-40	Wind speed 3	0001101011	107	10.7	Hz
41-50	Wind speed 4	0001111011	123	12.3	Hz
51-60	Wind speed 5	0001110110	118	11.8	Hz
61-70	Wind speed 6	0010000111	135	13.5	Hz
71-80	Wind speed 7	0010011000	152	15.2	Hz
81-90	Wind speed 8	0010010010	146	14.6	Hz
91-98	Direction 1	10101110	174	348	deg
99-106	Direction 2	10110000	176	352	deg
107-114	Direction 3	10110011	179	358	deg
115-122	Direction 4	00000010	2	4	deg
123-130	Direction 5	00000110	6	12	deg
131-138	Direction 6	00000011	3	6	deg
139-146	Direction 7	10110011	179	358	deg
147-154	Direction 8	10101111	175	350	deg

Bits	Field	Bits read	raw	Value	Unit
155-160	Debug	000000	0	0	-

Worked in full for the first wind speed: bits 11-20 are 0001010100 = 84, and $84 \times 0.1 + 0 = 8.4$ Hz. That is not yet a wind speed - section 3.4 finishes it, giving 5.82 m/s on a current anemometer. The eight speeds are the eight consecutive 15-second averages that make up the 2-minute period, oldest first: the wind rose from 8.4 to 15.2 Hz while the direction backed through north.



3. Five things that break naive decoders

3.1 All-ones means "no reading", not "maximum" - and only wind speed has it

When the firmware has no valid wind speed - the sensor failed, the value was NaN, or it fell outside the encodable range - it does not clamp and it does not omit the field. It writes every bit of that field to 1. The bit packer applies the same rule to any value it cannot represent.

Field	Bits	All-ones raw	Decodes to	Meaning
Wind speed, any of the eight	10	1023	102.3 Hz, about 62 m/s	No valid wind speed for that 15 s slot

A decoder that scales the all-ones pattern reports about 61 m/s - a plausible-looking storm - instead of a fault. Test for all-ones BEFORE scaling and emit null.

The rule does NOT apply to any other field in this payload. The index, the battery bin, the debug field and - importantly - the eight direction fields can all legitimately carry an all-ones pattern, and the direction fields never carry one as an error marker. The decoder in section 7 uses two different readers, `field()` and `plain()`, for exactly this reason.

The eight speed slots fail independently. One bad slot in a payload is normal near the encodable limit or during a sensor glitch; treat each slot on its own and do not discard the whole payload.

3.2 A failed wind direction is transmitted as 360 degrees

This is the trap that costs the most, because nothing about the payload looks wrong. When the direction sensor produces no samples for a 15-second slot the averaging routine returns its internal error value, 511 degrees. The range check that runs immediately afterwards sees 511 above the 360-degree maximum and **clamps it to 360** before the field is packed.

Situation	What the firmware does	What arrives
Direction sensor working, wind from the north	average rounds to 0, converted to 1 deg	raw 1 -> 2 deg
Direction sensor working, wind from 348 deg	average 348 deg	raw 174 -> 348 deg
No direction samples in the slot	error value 511, clamped to 360	raw 180 -> 360 deg
Direction above 360 for any reason	clamped to 360	raw 180 -> 360 deg

There is no invalid pattern for wind direction. A dead direction sensor reports a steady, perfectly valid-looking 360 degrees - due north - indefinitely, and a decoder cannot tell it apart from real northerly wind on the strength of one payload.

What a decoder can do is flag it and let the pattern across payloads settle the question. Only two of the 360 possible direction averages - 359 and 360 - quantise to a transmitted 360, so roughly one genuine reading in 180 lands there. A single slot reading 360 is therefore weak evidence of anything. Eight slots of 360 in one payload, repeated payload after payload while the wind speed still varies, is not weak evidence: real northerly wind wanders across 356, 358, 360 and 2, and a dead channel does not. The decoder in section 7 sets `directionSuspect` on any slot reading exactly 360 so a backend can apply that test itself.

3.3 The battery field is a 2-bit bin, and bin 0 is open-ended

Unlike every other product in the range, the HNWL payload reports a usable battery reading in every uplink rather than one bit that only means something across five consecutive indices. Two bits select one of four 200 mV bins. The bin index goes on the wire unscaled - the decoder applies the 3.5 V offset and the 0.2 V step itself.

Raw	Firmware test	Battery voltage	Report as
3	$v > 4.1 \text{ V}$	above 4.1 V	4.1 V or more
2	$v > 3.9 \text{ V}$	3.9 - 4.1 V	3.9 V
1	$v > 3.7 \text{ V}$	3.7 - 3.9 V	3.7 V
0	$v > 3.5 \text{ V}$, and everything below	at or below 3.7 V	3.5 V, with a low-battery flag

Raw 0 is ambiguous. The firmware's test cascade falls through to 0 both for 3.5 - 3.7 V and for any voltage below 3.5 V, so a nearly flat battery and a merely low one look identical. Treat raw 0 as "3.7 V or less" and raise a low-battery alert on it rather than plotting it as 3.5 V.

The other three bins are exact: the bin the firmware picks is the one the voltage actually falls in, so raw 2 really does mean 3.9 to 4.1 V. Read the value as the bottom of its bin, not the middle, and do not interpolate between consecutive payloads - the quantisation is 200 mV and the underlying reading does not move smoothly across a boundary.

3.4 The payload carries click frequency, not m/s

Every wind speed in this payload is transmitted as the anemometer click frequency in Hz, which is the number of cup revolutions per second. Converting it to m/s is the server's job, and the curve to use is a property of the anemometer fitted - it is never transmitted. The firmware itself never converts the payload values at all.

Anemometer	Wind speed from click frequency f [Hz]
Current (from 2023)	$v = -0.00065 * f^2 + 0.675 * f + 0.2$
Earlier (before 2023)	$v = 0.6335 * f + 0.3582$

Applying the wrong curve is a silent error. The two curves happen to agree around 60 Hz, which makes a mismatch easy to miss on a windy day, and diverge either side of it - by half a metre per second in the middle of the range and by 2.5 m/s at 100 Hz. Set the curve once, per device, to match the unit installed.

Click frequency	Current anemometer	Earlier anemometer	Difference
10 Hz	6.88 m/s	6.69 m/s	+0.19
20 Hz	13.44 m/s	13.03 m/s	+0.41
40 Hz	26.16 m/s	25.70 m/s	+0.46
60 Hz	38.36 m/s	38.37 m/s	-0.01
80 Hz	50.04 m/s	51.04 m/s	-1.00
102.3 Hz (field maximum)	62.45 m/s	65.17 m/s	-2.72

A cup anemometer does not turn below its starting threshold, around 0.3 Hz, so a frequency at or below that should be reported as calm rather than as the curve's offset term - otherwise every still night reads 0.2 m/s. The firmware applies the same 0.3 Hz floor internally.

One 0.1 Hz step is worth about 0.068 m/s at the bottom of the range and 0.054 m/s at the top, so a decoded speed is meaningful to roughly 0.05 m/s and no better. Do not present more precision than that.

3.5 Direction is quantised to 2 degrees

The direction fields are 8 bits at a resolution of 2 degrees, so only even values are ever transmitted. A genuine 357-degree average arrives as 358, and 11 degrees arrives as 12. The rounding is to nearest, so the error is at most 1 degree - but a decoder must not present the result as if it were the 1-degree resolution of the standard MeteoWind Gen2 payload, and a server must not treat a step from 348 to 350 as a real 2-degree veer.

North has two spellings in this payload. The averaging routine converts a computed 0 to 1 degree, which quantises to 2; the range check converts an out-of-range value to 360. Both mean north, and both are correct - see 3.2 for why the two are not interchangeable.



4. Port 1 - periodic measurement, 20 bytes

The only measurement uplink this firmware sends, and the only one it sends at all besides service messages. Fixed 2-minute period. Uses all 160 bits.

Bits	Field	W	Min	Res	Unit	Notes
1-8	Index	8	0	1	-	Revolving 0-255, incremented once per uplink. Not used for retransmission in this firmware
9-10	Battery bin	2	0	1	-	See 3.3 - apply $3.5 + 0.2 \times \text{raw volts}$ yourself
11-20	Wind speed 1	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 1. All-ones = invalid, see 3.1
21-30	Wind speed 2	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 2. All-ones = invalid, see 3.1
31-40	Wind speed 3	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 3. All-ones = invalid, see 3.1
41-50	Wind speed 4	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 4. All-ones = invalid, see 3.1
51-60	Wind speed 5	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 5. All-ones = invalid, see 3.1
61-70	Wind speed 6	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 6. All-ones = invalid, see 3.1
71-80	Wind speed 7	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 7. All-ones = invalid, see 3.1
81-90	Wind speed 8	10	0	0.1	Hz	Mean click frequency over 15-second sub-period 8. All-ones = invalid, see 3.1
91-98	Direction 1	8	0	2	deg	Mean direction over 15-second sub-period 1, 2-360. No invalid pattern, see 3.2
99-106	Direction 2	8	0	2	deg	Mean direction over 15-second sub-period 2, 2-360. No invalid pattern, see 3.2
107-114	Direction 3	8	0	2	deg	Mean direction over 15-second sub-period 3, 2-360. No invalid pattern, see 3.2
115-122	Direction 4	8	0	2	deg	Mean direction over 15-second sub-period 4, 2-360. No invalid pattern, see 3.2
123-130	Direction 5	8	0	2	deg	Mean direction over 15-second sub-period 5, 2-360. No invalid pattern, see 3.2
131-138	Direction 6	8	0	2	deg	Mean direction over 15-second sub-period 6, 2-360. No invalid pattern, see 3.2
139-146	Direction 7	8	0	2	deg	Mean direction over 15-second sub-period 7, 2-360. No invalid pattern, see 3.2
147-154	Direction 8	8	0	2	deg	Mean direction over 15-second sub-period 8, 2-360. No invalid pattern, see 3.2
155-160	Debug	6	0	1	-	Always 0 in this firmware. Not a bit field of status flags

4.1 The eight slots are consecutive 15-second sub-periods

This is the whole point of the HNWL payload. The station keeps a counter that advances every time it has accumulated 15 one-second samples; each advance writes one wind speed and one direction into the next array slot.

When the counter reaches 8 the payload is built and sent, and the counter wraps to 0.

Slot	Field pair	Covers	Relative to the uplink
1	Wind speed 1, Direction 1	the first 15 s of the period	-120 s to -105 s
2	Wind speed 2, Direction 2	the second 15 s	-105 s to -90 s
...	...	...	...
8	Wind speed 8, Direction 8	the last 15 s before transmission	-15 s to 0 s

Slot 1 is the oldest and slot 8 the newest. All eight are always fresh: the payload is emitted exactly when the eighth slot has been written, so a stale slot from a previous period can never appear. There is no period average, no gust, no minimum and no standard deviation in this payload - compute whatever of those you need from the eight slots, skipping the invalid ones rather than treating them as zero.

4.2 Timestamping the slots

The payload carries no timestamps. Take the network's receive time as the end of slot 8 and work backwards in 15-second steps. That is accurate to the transmission latency, which for a Class A uplink is small and constant.

The 15 seconds are nominal, not guaranteed. A sub-period ends when 15 one-second samples have been accumulated, and those samples are only counted while the measurement task is processing events, so a sub-period can stretch slightly under load. Over a 2-minute period the drift is small, but do not build anything that depends on the slot boundaries being exactly 15.000 s apart, and re-anchor on the receive time of every payload rather than extrapolating a slot clock forwards.

4.3 Wind above 102.3 Hz is indistinguishable from a fault

The 10-bit field at 0.1 Hz tops out at 102.3 Hz - about 62 m/s on a current anemometer. The bit packer does not clamp values above that: it writes all-ones, the same pattern the firmware uses for a failed reading. A genuine hurricane-force gust and a dead anemometer produce the identical field.

So does a value of exactly 102.3 Hz, which lands on raw 1023 legitimately. In practice this matters only in the top 1 percent of the range; report the slot as null and let the neighbouring slots and the site's climatology settle which it was.

4.4 There is no transmit spreading

The standard MeteoWind Gen2 derives a 0 to 29 second delay from the payload contents so that a fleet of stations does not transmit in unison. That calculation is disabled in this firmware - the delay is always zero and the station transmits as soon as the payload is built.

For a decoder this is good news: the uplink interval is a steady 2 minutes rather than 2 minutes plus a variable delay, so a missed payload is easier to detect. For a fleet it means the stations must be staggered at commissioning time, because the firmware will not stagger them.

5. Port 2 - service messages, 12 bytes

Three different messages share port 2 and all three are 12 bytes long. They are sent only in response to downlink command 120, never on their own schedule.

MW0S, MW1S and MW2S are all 12 bytes. Read the first 2 bits to tell them apart: 0 is MW0S, 1 is MW1S, 2 is MW2S. Do not branch on length.

5.1 MW1S - device identity

Firmware version, LoRaWAN address, measurement period and averaging mode. Requested with downlink 120 and payload 01. Uses 72 of the 96 bits; bits 73-96 are zero padding, not a field.

Bits	Field	W	Notes
1-2	Message type	2	Always 1 for MW1S. This is the discriminator

Bits	Field	W	Notes
3-7	Hardware type	5	0 test, 1 MeteoHelix, 2 MeteoWind, 3 MeteoRain, 4 MeteoAG. Always 2 here
8-10	FW major	3	Firmware version, first component
11-16	FW minor	6	Second component. Render zero-padded to 2 digits
17-24	FW patch	8	Third component. Render zero-padded to 3 digits
25-56	Device address	32	LoRaWAN DevAddr. Render as 8 hex digits
57-64	Debug	8	Mirrors the averaging mode in this firmware. Do not rely on it
65-71	Measurement period	7	Minutes between periodic uplinks. Always 2 - see below
72	Payload type	1	0 scalar averaging, 1 vector averaging. Not settable in this firmware
73-96	(padding)	24	Zero fill, not a field

The three version components are separate fields, so 1, 2, 4 renders as 1.02.004 - zero-pad the minor to two digits and the patch to three to match how the station names its own version.

The measurement period field is fixed. The firmware defines the minimum, maximum and default uplink period as 2 minutes, and downlink 132 is rejected, so this field always reads 2. Do not build a scheduler that expects it to change, and do not read a different value as evidence of a different configuration - it would mean corrupt EEPROM.

5.2 MW2S - alarm settings readback

The alarm settings stored in EEPROM. Requested with downlink 120 and payload 02. Uses 32 of the 96 bits; bits 33-96 are zero padding.

Bits	Field	W	Min	Res	Unit	Notes
1-2	Message type	2	0	1	-	Always 2 for MW2S
3-11	Alarm snooze	9	0	120	s	Hold-off after an alarm, 120-61320 s
12	Alarm mode	1	0	1	-	1 high alarm, 0 low alarm
13-19	Low threshold	7	0	1	Hz	LOW COMES FIRST in this message
20-26	High threshold	7	0	1	Hz	0-127 Hz
27-32	Debug	6	0	1	-	Always 0 in this firmware
33-96	(padding)	64	-	-	-	Zero fill, not a field

These settings do nothing. The alarm is not implemented in HNWL firmware - see 6.1 - and the downlink that would change them is rejected - see 6.3. MW2S reports the values held in EEPROM, and those values are never acted on and can never be changed over the air. Decode the message so it is not mistaken for something else, but do not present the thresholds to a user as if the station were armed.

In practice every station in the field returns the same message, the production defaults: snooze 600 s, high-alarm mode, low threshold 0 Hz, high threshold 47 Hz. That is the vector 80B00BC0000000000000000000 in section 7.1.

5.3 MW0S - position and inclination, not implemented

MW0S is transmitted, and every value in it is a fixed placeholder compiled into the firmware. No GPS receiver and no inclinometer are fitted to this product. Every station answers with the same coordinates, the same satellite count and the same inclination, indefinitely.

It is listed here only so that a decoder recognises the message and discards it rather than storing a fleet of stations at one address in Slovakia. The layout below is what the bits contain; do not build features on it. Note also that latitude and longitude are encoded with a minimum of zero, so the format cannot represent southern or western positions at all.

Bits	Field	W	Min	Res	Unit / notes
1-2	Message type	2	0	1	Always 0 for MW0S
3-28	Latitude	26	0	0.00001	deg
29-53	Longitude	25	0	0.00001	deg
54-55	Fix status	2	0	1	bit 0 GNSS, bit 1 DGPS
56-58	Satellites	3	0	1	6 means 6 or more
59-66	HDOP	8	0	0.1	-
67-74	Inclination X	8	0	0.25	deg
75-82	Inclination Y	8	0	0.25	deg
83-90	Inclination Z	8	0	0.25	deg
91-96	Debug	6	0	1	-



6. What this firmware does not send

The HNWL firmware is the standard MeteoWind Gen2 firmware with several features deliberately switched off. The code for them is still present but the call sites are commented out, so the features are not merely unused - they cannot be triggered. A decoder written against the standard product will wait forever for messages that never arrive, and a backend will queue downlinks that are silently ignored.

6.1 No alarm payload, no port 3

The station never sends an alarm uplink. The payload builder for it is compiled in and the layout below is what it would produce, but the call that would send it is commented out in the radio task, and nothing in the firmware ever fills the alarm result structure. There is no wind speed threshold monitoring of any kind in this build.

Bits	Field	W	Min	Res	Unit	Notes
1-5	Index	5	0	1	-	Revolving 0-31
6-16	3s gust	11	0	0.1	Hz	Absolute, not an addition
17-24	1s gust addition	8	0	0.1	Hz	Add to the 3 s gust
25-33	Direction of gust	9	0	1	deg	1-360
34-40	Time of 1s gust	7	0	5	s	Seconds from the start of the period
41-48	Alarm flags	8	0	1	-	Bit 0: 1 = high alarm, 0 = low alarm

This table is here so that nobody rediscovers the 6-byte layout and concludes the feature works. If a payload ever arrives on port 3 from a station you believe is running HNWL firmware, the station is not running HNWL firmware. Treat it as a device-identification error, not as an alarm.

6.2 No retransmission, no port 4

Payloads are not logged to EEPROM and cannot be replayed. Downlink 108 (retransmit a stored periodic payload) and downlink 109 (retransmit a stored alarm payload) are both accepted by the radio, logged to the debug console as not supported, and discarded. Nothing is ever sent on port 4.

The practical consequence is that a payload lost on the air is lost. The index field still increments so gaps are detectable, but there is no way to recover the missing measurement - which for this product means eight 15-second slots, two minutes of data.

6.3 No alarm-settings downlink, no port 10

A 3-byte command on port 10 sets the alarm snooze, mode and thresholds on the standard MeteoWind Gen2. In HNWL firmware the handler logs the command and returns without acting on it. The settings in EEPROM keep their production defaults for the life of the device.

There is no reply on port 10 in either firmware, so a rejected command looks exactly like a delivered one. The only way to tell that nothing happened is to request MW2S and see the old values - which, on HNWL, is the only outcome there is.

6.4 The downlink commands that do work

These are the commands the HNWL firmware acts on. All are Class A: a downlink queued on the network server is delivered after the station's next uplink, so up to 2 minutes away. Queue one command at a time and wait for it to be delivered.

Port	Command	Payload	Effect	Reset
100	Configure OTAA	24 B - AppKey + AppEUI	Saves the new keys, then reboots	Yes
102	Configure RX2	5 B	Saves RX2 frequency and data rate, then reboots	Yes

Port	Command	Payload	Effect	Reset
103	Configure ADR	1 B	0 off, 1 on. Saves, then reboots	Yes
104	Set data rate	1 B	Data rate 0-7 depending on region. Saves, then reboots	Yes
106	Set confirmed uplinks	1 B	Sets the confirmation mode in RAM only. It is not written to EEPROM, so a reboot restores the previous setting	No
107	Rejoin	1 B	Waits, then rejoins the network. The argument is ignored - any payload on this port triggers a rejoin, not just 01	No
110	Reset device	1 B	Reboots if the argument is 1	Yes
111	Rejoin switch and counter	3 B	Byte 1: 00 off, 01 on, 02 report to port 50. Bytes 2-3: uplinks between forced rejoins, 5 to 4095	No
120	Service message request	1 B	00 sends MW0S, 01 sends MW1S, 02 sends MW2S on port 2	No
130 / 131	Time sync, absolute / relative	4 B hex	Sets the real-time clock, and only if the last uplink was a periodic payload. No payload carries a timestamp, so this affects the debug log and nothing a decoder sees	No
200	Diagnostic	-	Prints diagnostics to the debug UART only	No

Command 105 is accepted by the radio driver and does nothing. Commands 108, 109, 121 and 132 are explicitly rejected - see 6.1 to 6.3 and 5.1. Command 132 deserves a second look: the driver does parse it, but the only value its range check accepts is 2, which is the period the station already uses, and the task-level handler does not persist the result. There is no argument that changes anything.

The customer-facing command list is narrower still - ports 100, 104, 107, 110, 111 and 120. The others in the table are present in the firmware but are not part of the documented interface for this product.

Port 50 carries the replies. Command 111 with argument 02 answers there with the rejoin switch, the configured counter and the current count as hex; the replies carry no command identifier, so correlate them with the command you sent.

7. Reference decoder

The decoder below is supplied alongside this document as the file `meteowind_gen2_hnwl_decoder.js`. Use that file rather than retyping from these pages.

It is written in ES5 with no dependencies, so it pastes directly into a The Things Stack v3 or ChirpStack v4 payload-formatter box; both call `decodeUplink({bytes, fPort})`. Set ANEMOMETER at the top to match the unit before using it. The decoder returns the transmitted frequencies as well as the converted speeds, so the intermediate values stay available for checking, and it returns the eight slots both as an array of objects with time offsets and as three flat columns.

It was verified field by field against firmware 1.02.004 using the test vectors in section 7.1, including the two failure cases, and against the company's own bit-string calculator. Run them through your own decoder before trusting it.

7.1 Test vectors

Feed these through any decoder before trusting it. They were generated from an encoder that mirrors the firmware's bit packer. Speeds in m/s use the current anemometer curve.

Port	Payload (hex)	Expected
1	0385416C6B1EC7621C9824ABAC2CC08180ECEBC0	index 3, battery 3.9-4.1 V, speeds 8.4 / 9.1 / 10.7 / 12.3 / 11.8 / 13.5 / 15.2 / 14.6 Hz (5.82 to 10.31 m/s), directions 348 / 352 / 358 / 4 / 12 / 6 / 358 / 350 deg. Rising wind backing through north
1	073FFFFFFFFFFFFFFFFFED2D2D2D2D2D2D00	index 7, battery raw 0 (3.7 V or less), all eight speeds null - anemometer failure - and all eight directions 360 deg, which here is the clamp of section 3.2, not real northerly wind
1	00C000000000000000000000004040404040404040	index 0, battery above 4.1 V, dead calm: all eight speeds 0 Hz and 0 m/s, all eight directions 2 deg. This is the other spelling of north - see 3.5
1	294D7390C9314EA3E4DE34AD2D2D2D2D2D2D00	index 41, battery 3.7-3.9 V, speeds 19.7 to 24.9 Hz (13.25 to 16.60 m/s) - all valid - and all eight directions 360 deg. The silent failure of 3.2: the anemometer is fine and the direction channel is dead
1	C87B6F8FFFFFFFFDCB1FB2A1E1E1E1E1E1E1E1C0	index 200, speeds 95.0 / 99.5 / null / null / null / 88.2 / 79.9 / 71.4 Hz. Slots 3 to 5 exceeded 102.3 Hz and are indistinguishable from a fault - see 4.3. Directions all 270 deg
1	02C000000028E6012C080026AC83834B82814141	the vector from the company bit-string calculator: index 2, battery raw 3, speeds 0 / 0 / 0 / 16.3 / 60.8 / 7.5 / 0.8 / 0 Hz, directions 308 / 356 / 28 / 26 / 92 / 20 / 10 / 10 deg. Its debug field is 1; a real station always sends 0
2	4442042700BE440004000000	MW1S, MeteoWind, firmware 1.02.004, DevAddr 2700BE44, 2 minute period, scalar averaging
2	80B00BC0000000000000000000	MW2S, the production defaults every HNWL station reports: snooze 600 s, high-alarm mode, low threshold 0 Hz, high threshold 47 Hz. Inert - see 5.2
2	049780C0D0D5975EC8028080	MW0S, the fixed placeholder values of section 5.3. Recognise and discard

Two negative vectors are worth running as well. A 14-byte payload on port 1 is a standard MeteoWind Gen2 periodic payload and must be rejected, not decoded - 039348F8709030ED7A08B1A4294C is one. Anything at all on port 3, 4 or 10 must be rejected too; the decoder in 7.2 names the port in the error so the cause is obvious in a log.

7.2 Source

```
// =====
// MeteoWind IoT Pro Gen2 HNWL - LoRaWAN uplink payload decoder
// Barani Design Technologies
//
```

```

// For firmware 1.02.004. See the Payload Decoding Guide for the field
// layouts and the reasoning behind them.
//
// Works as-is as a TTN v3 / ChirpStack v4 JavaScript codec: paste the whole
// file into the payload-formatter box. decodeUplink({bytes, fPort}) is the
// entry point both stacks call.
//
// Ports: 1 periodic (20 B), 2 service MW0S/MW1S/MW2S (12 B),
//        50 command reply (raw hex).
//
// This is the HNWL variant. It does NOT send port 3 (alarm) or port 4
// (retransmission), and it does NOT accept the port 10 alarm-settings
// downlink - those code paths are compiled out. The periodic payload is a
// different message from the standard MeteoWind Gen2 one: 20 bytes carrying
// eight 15-second sub-period averages instead of 14 bytes of gust
// statistics. A standard Gen2 decoder will not decode it.
// =====
// ANEMOMETER: which cup assembly is fitted. The payload carries click
// frequency in Hz, never m/s, and the curve is not transmitted.
// 'current' - units made from 2023 onward (quadratic curve)
// 'legacy' - units made before 2023 (linear curve)
var ANEMOMETER = 'current';
// Frequency at or below which the cups are not turning. Report calm rather
// than the offset term of the curve.
var CALM_HZ = 0.3;
// Nominal length of one sub-period, in seconds. Eight of them make the uplink.
var SLOT_SECONDS = 15;
// ----- bit reader
function Reader(bytes) {
  this.b = bytes;
  this.pos = 0;
}
Reader.prototype.bits = function (n) {
  var v = 0;
  for (var i = 0; i < n; i++) {
    var byteNum = (this.pos + i) >> 3;
    var bitNum = 7 - ((this.pos + i) & 7);
    v = v * 2 + ((this.b[byteNum] >> bitNum) & 1);
  }
  this.pos += n;
  return v;
};
// Read a field and scale it. Returns null when every bit is set, which the
// firmware uses to mean "no valid reading" - not a maximum value.
function field(r, bits, min, res) {
  var raw = r.bits(bits);
  if (raw === Math.pow(2, bits) - 1) return null;
  return round4(raw * res + min);
}
// Same, but the all-ones pattern is a legitimate value (counters, flags, IDs).
function plain(r, bits, min, res) {
  var raw = r.bits(bits);
  if (min === undefined) return raw;
  return round4(raw * res + min);
}
function round4(x) {
  return Math.round(x * 10000) / 10000;
}
// ----- anemometer curves
// Click frequency [Hz] -> wind speed [m/s]. Property of the cup assembly.
function speedFromHz(f) {
  if (f === null) return null;
  if (f <= CALM_HZ) return 0;
  if (ANEMOMETER === 'legacy') return round4(0.6335 * f + 0.3582);
  return round4(-0.00065 * f * f + 0.675 * f + 0.2);
}
// ----- battery, 2 bits
// The bin index goes on the wire unscaled. Bin n covers 200 mV starting at
// 3.5 V - except bin 0, which is open-ended downwards: it means both
// "3.5 to 3.7 V" and "anything below 3.5 V". See the guide.
function batteryFromBin(bin) {
  var lower = 3.5 + 0.2 * bin;
  return {
    bin: bin,
    min_V: bin === 0 ? null : round4(lower),
    max_V: bin === 3 ? null : round4(lower + 0.2),
    label: bin === 0 ? 'at or below 3.7 V (includes a flat battery)'
      : bin === 3 ? 'above 4.1 V'
      : round4(lower) + ' - ' + round4(lower + 0.2) + ' V'
  };
};
// ----- port 1
function decodePeriodic(r) {
  var o = {}, i, hz, deg;
  o.index = plain(r, 8);
  o.battery = batteryFromBin(plain(r, 2));
  // Eight consecutive 15 s sub-period averages, oldest first.
  var freq = [], dir = [];
  for (i = 0; i < 8; i++) freq.push(field(r, 10, 0, 0.1)); // all-ones = invalid
  for (i = 0; i < 8; i++) dir.push(plain(r, 8, 0, 2)); // never all-ones
}

```

```

o.debug = plain(r, 6);
o.slots = [];
for (i = 0; i < 8; i++) {
  hz = freq[i];
  deg = dir[i];
  o.slots.push({
    slot: i,
    offsetStart_s: -(8 - i) * SLOT_SECONDS, // 0 = oldest 15 s of the period
    offsetEnd_s: -(7 - i) * SLOT_SECONDS, // relative to the uplink instant
    frequency_Hz: hz,
    windSpeed_ms: speedFromHz(hz),
    // 360 is north AND the value the firmware substitutes for a failed
    // direction reading. It is not possible to tell them apart.
    direction_deg: deg === 0 ? 360 : deg,
    directionSuspect: deg === 360
  });
}
// Flat arrays, for backends that would rather have columns than objects.
o.frequency_Hz = freq;
o.windSpeed_ms = [];
o.direction_deg = [];
for (i = 0; i < 8; i++) {
  o.windSpeed_ms.push(o.slots[i].windSpeed_ms);
  o.direction_deg.push(o.slots[i].direction_deg);
}
// Period summary. Nulls are skipped, not treated as zero.
var sum = 0, n = 0, mx = null;
for (i = 0; i < 8; i++) {
  if (freq[i] === null) continue;
  sum += freq[i]; n++;
  if (mx === null || freq[i] > mx) mx = freq[i];
}
o.validSlots = n;
o.periodMean_Hz = n ? round4(sum / n) : null;
o.periodMean_ms = speedFromHz(o.periodMean_Hz);
o.periodMax_Hz = mx;
o.periodMax_ms = speedFromHz(mx);
return o;
}
// ----- port 2
var DEVICE_TYPE = ['test', 'MeteoHelix', 'MeteoWind', 'MeteoRain', 'MeteoAG'];
function decodeMw1s(r) {
  var o = { messageType: 'MW1S' };
  var hw = plain(r, 5);
  o.hardwareTypeValue = hw;
  o.hardwareType = DEVICE_TYPE[hw] !== undefined ? DEVICE_TYPE[hw] : 'unknown';
  var maj = plain(r, 3), min = plain(r, 6), pat = plain(r, 8);
  o.firmware = maj + '.' + pad2(min) + '.' + pad3(pat);
  o.devAddr = hex8(plain(r, 32));
  o.debug = plain(r, 8); // mirrors payloadType in this firmware
  o.measurementPeriod_min = plain(r, 7); // always 2 - the period is not settable
  o.payloadType = plain(r, 1); // 0 scalar, 1 vector - not settable
  // Remaining 24 bits are zero padding - do not read them.
  return o;
}
function decodeMw2s(r) {
  var o = { messageType: 'MW2S' };
  o.allAlarmsSnooze_s = plain(r, 9, 0, 120);
  o.alarmMode = plain(r, 1) ? 'high' : 'low';
  o.lowThreshold_Hz = plain(r, 7); // LOW COMES FIRST - see the guide
  o.highThreshold_Hz = plain(r, 7);
  o.debug = plain(r, 6);
  o.note = 'the alarm is not implemented in HNWL firmware - these settings ' +
    'are stored and reported but never act on anything';
  // Remaining 64 bits are zero padding - do not read them.
  return o;
}
function decodeMw0s(r) {
  var o = { messageType: 'MW0S' };
  o.latitude_deg = plain(r, 26, 0, 0.00001);
  o.longitude_deg = plain(r, 25, 0, 0.00001);
  var fix = plain(r, 2);
  o.fixStatus = { value: fix, gnss: (fix & 1) === 1, dgps: (fix & 2) === 2 };
  o.satellites = plain(r, 3); // 6 means 6 or more
  o.hdop = plain(r, 8, 0, 0.1);
  o.inclinationX_deg = plain(r, 8, 0, 0.25);
  o.inclinationY_deg = plain(r, 8, 0, 0.25);
  o.inclinationZ_deg = plain(r, 8, 0, 0.25);
  o.debug = plain(r, 6);
  return o;
}
function pad2(n) { return (n < 10 ? '0' : '') + n; }
function pad3(n) { return (n < 10 ? '00' : n < 100 ? '0' : '') + n; }
function hex8(n) {
  var s = (n >>> 0).toString(16).toUpperCase();
  while (s.length < 8) s = '0' + s;
  return s;
}
// ----- dispatch
function decodeUplink(input) {

```



```

var b = input.bytes, port = input.fPort;
var r = new Reader(b);
var data, warnings = [], i;
if (port === 1) {
  if (b.length !== 20) return err('port 1 needs 20 bytes, got ' + b.length +
    (b.length === 14 ? ' - 14 bytes is the standard MeteoWind Gen2 payload, ' +
      'not the HNWL one' : ''));
  data = decodePeriodic(r);
  if (data.validSlots === 0)
    warnings.push('every wind speed slot is invalid - check the anemometer');
  for (i = 0; i < 8; i++) {
    if (data.slots[i].directionSuspect) {
      warnings.push('one or more direction slots read exactly 360 deg, which ' +
        'is both north and the substitute for a failed reading');
      break;
    }
  }
} else if (port === 2) {
  // MW0S, MW1S and MW2S are all 12 bytes - the leading 2 bits tell them apart.
  if (b.length !== 12) return err('port 2 needs 12 bytes, got ' + b.length);
  var type = r.bits(2);
  if (type === 0) {
    data = decodeMw0s(r);
    warnings.push('MW0S carries fixed placeholder values - do not store them');
  } else if (type === 1) data = decodeMw1s(r);
  else if (type === 2) data = decodeMw2s(r);
  else return err('unknown service message type ' + type);
} else if (port === 50) {
  // Configuration command replies. Correlate with the command just sent -
  // these carry no command ID.
  return { data: { commandReply: bytesToHex(b) },
    warnings: ['port 50 reply - identify it from the command you sent'] };
} else if (port === 3 || port === 4 || port === 10) {
  return err('port ' + port + ' is not used by HNWL firmware - a payload here ' +
    'did not come from a MeteoWind Gen2 HNWL station');
} else {
  return err('unhandled port ' + port);
}
return { data: data, warnings: warnings };
}
function err(m) { return { data: {}, errors: [m] }; }
function bytesToHex(b) {
  var s = '';
  for (var i = 0; i < b.length; i++) {
    var h = b[i].toString(16).toUpperCase();
    s += (h.length < 2 ? '0' : '') + h;
  }
  return s;
}
// Node/CommonJS export - harmless in TTN and ChirpStack, which ignore it.
if (typeof module !== 'undefined') {
  module.exports = { decodeUplink: decodeUplink, speedFromHz: speedFromHz };
}

```

8. Migrating a standard MeteoWind Gen2 decoder

There is no migration path in the usual sense: the periodic payloads have nothing in common past the index field, so an HNWL decoder is a new decoder rather than a patched one. What follows is the list of assumptions a standard Gen2 decoder carries that are wrong here.

Assumption from the standard Gen2	What is true on HNWL
Port 1 is 14 bytes	20 bytes. Length-check it - this is the cheapest possible way to catch a misconfigured decoder before it publishes wrong data
The payload carries one period average and two gusts	It carries eight 15-second averages and no gust at all. Nothing in the payload is a gust; the closest equivalent is the largest of the eight slots, which is a 15 s mean, not a 3 s or 1 s peak
Gust fields are additions to the average	There are no gust fields. Every speed field is absolute
Direction 511 means the sensor failed	Direction has no invalid pattern. A failed sensor reports 360 - see 3.2
Direction resolution is 1 degree	2 degrees, even values only - see 3.5
The battery bit needs five consecutive indices	The battery is a 2-bit bin, usable in every payload, with an open-ended bottom bin - see 3.3
The uplink period is 2 to 10 minutes and settable	Fixed at 2 minutes. Downlink 132 is rejected and MW1S always reports 2
Uplinks are spread by 0 to 29 seconds	No spreading - see 4.4
Alarms arrive on port 3	The alarm is not implemented - see 6.1
Lost payloads can be requested back on port 4	No retransmission - see 6.2
Alarm thresholds can be set on port 10	The command is rejected - see 6.3
The last 5 bits of port 1 are a turbulence index	There is no turbulence field. The last 6 bits are a debug field and are always 0

The service messages are the one part that does carry over. MW0S, MW1S and MW2S have identical layouts on both products, and an existing port 2 decoder works unchanged - with the caveats of section 5 about what the values mean here.

9. Decoder checklist

#	Check
1	Read bit ranges, not bytes
2	Reject port 1 payloads that are not exactly 20 bytes - 14 bytes is the standard Gen2 payload
3	Test each of the eight wind speed fields for all-ones and emit null - do not scale it
4	Do not apply the all-ones rule to the index, the battery bin, the direction fields or the debug field
5	Treat the eight speed slots as failing independently - one bad slot does not invalidate the payload
6	Flag any direction slot reading exactly 360 deg as suspect, and eight of them as a dead direction sensor
7	Convert Hz to m/s with the curve for the anemometer fitted, and set speeds at or below 0.3 Hz to calm
8	Do not present decoded speed to better than about 0.05 m/s or direction to better than 2 deg
9	Read the battery as a bin: raw 0 means 3.7 V or less and deserves a low-battery alert
10	Timestamp slot 8 at the receive time and step back 15 s per slot; re-anchor on every payload

#	Check
11	Compute any period average, maximum or gust yourself from the valid slots only
12	Branch port 2 on the leading 2 bits, not on length - all three service messages are 12 bytes
13	Recognise MW0S and discard it - its contents are fixed placeholders
14	Do not treat MW2S alarm thresholds as active - the alarm is not implemented
15	Expect exactly 2 minutes between uplinks, with no transmit spreading, and a fixed period of 2 in MW1S
16	Reject anything on ports 3, 4 and 10 - this firmware neither sends nor accepts them

